

Resource Adaptivity at Task-Level

Jonas Posner[✉]

University of Kassel, Kassel, Germany, jonas.posner@uni-kassel.de

User Experience Abstract. Traditionally, resource management in supercomputers is *static*: jobs request a fixed set of resources and retain them throughout execution. This static approach has significant drawbacks: resource managers are inflexible in scheduling jobs and programs cannot adjust resources to meet changing needs of different phases. This leads to inefficient resource utilization and reduced overall performance.

Adaptive Resource Management enables dynamic control of resources, significantly improving supercomputer efficiency. However, resource adaptivity requires support from both the resource manager and programs. If so, running jobs can change their resource allocations by integrating or releasing resources initiated by the resource manager (*malleable*) or by the jobs themselves (*evolving*). Jobs that can do both are called *adaptive*.

Resource adaptivity is currently not widely used in practice. Reasons include a significantly increased programming complexity for adaptive programs compared to traditional rigid ones. Moreover, widely used resource managers, such as SLURM, and inter-node programming models, such as MPI, offer only limited adaptivity functionalities. Existing approaches often require extensive code modifications and focus on iterative computations providing natural points for resource changes.

Asynchronous Many-Task (AMT) programming is a promising approach to enhance programmer productivity, balance dynamic and irregular workloads (having unpredictable computation patterns), and provide resource adaptivity with minimal code additions. In AMT, computations are split into many small *tasks*, which the runtime system dynamically assigns to *workers*. AMT's transparent resource management makes it ideal for adaptivity: programs can seamlessly adapt to resource changes by redistributing tasks and data. However, there is still a lack of AMT runtime systems that provide adaptivity in an uncomplicated and effective way.

This work aims to bridge this gap by proposing an adaptivity extension to the AMT system “*APGAS for Java*”. APGAS extends the Partitioned Global Address Space (PGAS) model by incorporating asynchronous task capabilities. While the original APGAS supports changes in the number of processes, it does so in a very rudimentary manner. The following summarizes the contributions of previous work [1], [2], [3], [4]:

Malleability and Evolving Techniques. We propose and implement innovative techniques for both malleability and evolving capabilities. With our APGAS extension, programmers enable adaptivity by specifying actions required when changing resources, i.e., before and after removing or adding a process. Runtime adjustments, including process initiation and termination, are managed automatically. Malleable programs react to externally initiated resource changes, while evolving

programs automatically initiate resource changes themselves in response to their computational loads at runtime. While other approaches often rely on synchronization points, our techniques do not require interrupting the computation to change resources.

Automatic Load Detection Heuristics. We propose two heuristics for automatic detection of computational loads: one using CPU load and one task loads. They determine when to start a new process (there is sufficient load) or terminate a process (there are underutilized nodes). This is particularly beneficial for irregular and dynamic workloads with unpredictable fluctuations in resource needs at runtime.

Practical Usability Demonstration. We demonstrate the practical usability of our extension by adapting the *Global Load Balancing (GLB)* library and its benchmarks. Additionally, we develop a new configurable benchmark designed to evaluate performance under highly irregular and dynamic workloads.

Communication with Resource Managers. We propose a generic communication interface that enables communication between programs and resource managers. Malleable programs react to *shrink/grow* messages received from the resource manager, while evolving programs can return resources and request new resources from the resource manager. Communication can be made at any time and is not restricted to any fixed points.

Resource Manager. We develop a prototype resource manager including five job scheduling algorithms supporting rigid, moldable, malleable, and evolving jobs.

Evaluation. We perform real-world experiments to evaluate performance. Changing resources causes an execution time overhead of approximately 0.5 sec for moving tasks/data and 3.5 sec for starting/terminating processes. The load detection heuristics efficiently identify load changes at runtime. Execution of 100% *malleable* jobs with our malleable resource manager, compared to 100% rigid jobs, showed: 13% decrease in job batch makespan, 20% increase in node utilization, and 3% decrease in job turnaround time. Execution of 100% *evolving* jobs showed: 23% decrease in job batch makespan, 4% increase in node utilization, and 29% decrease in job turnaround times.

Conclusions. We demonstrated that both malleable and evolving capabilities can be effectively implemented in AMT. This work-in-progress has some limitations, and we are currently working on a new implementation using MPI Sessions.

REFERENCES

- [1] P. Finnerty, J. Posner *et al.*, “On the Performance of Malleable APGAS Programs and Batch Job Schedulers,” *SN Computer Science*, 2024.
- [2] J. Posner, R. Goebel *et al.*, “Evolving APGAS Programs: Automatic and Transparent Resource Adjustments at Runtime,” in *WAMTA*, 2024.
- [3] J. Posner, “The Impact of Evolving APGAS Programs on HPC Clusters,” in *Euro-Par DynResHPC*, 2024, to appear.
- [4] J. Posner and P. Finnerty. (2024) <https://github.com/ProjectWagomu>.